PhosphoDisco: A Toolkit for Co-regulated Phosphorylation Module Discovery in Phosphoproteomic Data

Tobias Schraink[1,2,3,‡], Lili Blumenberg[1,2,‡], Grant Hussey[1,2,3,‡], Sabrina George[1,2], Brecca Miller[1,2], Nithu Mathew[1,2], Tania J. González-Robles[1,2,3], Vladislav Sviderskiy[4], Thales Papagiannakopoulos[4], Richard Possemato[4], David Fenyö[2,3], and Kelly V. Ruggles[1,2,*]

Kinases are key players in cancer-relevant pathways and are the targets of many successful precision cancer therapies. Phosphoproteomics is a powerful approach to study kinase activity and has been used increasingly for the characterization of tumor samples leading to the identification of novel chemotherapeutic targets and biomarkers. Finding co-regulated phosphorylation sites which represent potential kinase-substrate sets or members of the same signaling pathway allows us to harness these data to identify clinically relevant and targetable alterations in signaling cascades. Unfortunately, studies have found that databases of co-regulated phosphorylation sites are only experimentally supported in a small number of substrate sets. To address the inherent challenge of defining co-regulated phosphorylation modules relevant to a given dataset, we developed PhosphoDisco, a toolkit for determining co-regulated phosphorylation modules. We applied this approach to tandem mass spectrometry based phosphoproteomic data for breast and non-small cell lung cancer and identified canonical as well as putative new phosphorylation site modules. Our analysis identified several interesting modules in each cohort. Among these was a new cell cycle checkpoint module enriched in basal breast cancer samples and a module of PRKC isozymes putatively co-regulated by CDK12 in lung cancer. We demonstrate that modules defined by PhosphoDisco can be used to further personalized cancer treatment strategies by establishing active signaling pathways in a given patient tumor or set of tumors, and in providing new ways to classify tumors based on signaling activity.

Protein phosphorylation results in conformational changes, leading to changes in protein activity, substrate affinity, and degradation. This process is regulated by enzymatic kinases and phosphatases that catalyze the transfer of phosphate between their substrates. Subsequently, the activity of these enzymes results in the activation or deactivation of signaling pathways that drive different cellular processes such as cell growth, apoptosis, and differentiation. Improper regulation of these pathways can lead to severe disease states such as cancer. Although protein kinase genes account for only 2% of human genes, up to 30% of all human proteins can be modified by kinase activity (1, 2). Therefore, exploring the role of phosphoproteins and the mechanisms of kinases is vital in contributing to our understanding of cancer biology. Further, kinases are eminently targetable and represent some of the most successful personalized cancer therapeutics developed to date (3–7).

Quantitation of the phosphoproteome by mass spectrometry (MS) provides a particularly useful perspective on signaling patterns and vulnerabilities in cancer and can provide a personalized view of aberrations in potentially targetable pathways. However, comprehensively discerning the activity of phosphorylation signaling pathways is challenging because phosphorylation is often context-dependent and significantly modified in cancer. To address this challenge, the Clinical Proteomics Tumor Analysis Consortium (CPTAC) has conducted proteogenomic characterization of several cancer types by performing DNA and RNA sequencing, as well as MS-based proteomic and phosphoproteomic analysis (8–17). These efforts have generated rich high-dimensional data sets which have been used to generate and test novel hypotheses. However, identifying relevant pathways from these large-scale omics studies can be difficult, largely due to their high dimensionality and co-linearity. Further, it has been shown that the correlation between kinase abundance and known substrate sets in multiple CPTAC data sets is close to what

From the [1]Division of Precision Medicine, Department of Medicine, [2]Institute for Systems Genetics, [3]Department of Biochemistry and Molecular Pharmacology, and [4]Department of Pathology, New York University Grossman School of Medicine, New York, New York, USA
‡Indicates equal contributions.
*For correspondence: Kelly Ruggles, Kelly.Ruggles@nyulangone.org.

© 2023 THE AUTHORS. Published by Elsevier Inc on behalf of American Society for Biochemistry and Molecular Biology.
This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).
https://doi.org/10.1016/j.mcpro.2023.100596

would be expected by chance (18) and independently identified associations between phosphopeptides and kinases have minimal overlap with these curated sets (19). There are many reasons why kinase-substrate sets curated from myriad sources would not be reflected in a particular cancer dataset, but the challenge of defining co-regulated phosphosites in cancer data remains.

To better define these co-regulated modules in patient samples, we developed a computational toolkit, PhosphoDisco, for the analysis of tumor phosphoproteomic and proteomic data. We applied PhosphoDisco to data from a cohort of breast cancer (BRCA) tumors, and a combined data set of lung squamous cell carcinoma (LSCC) and lung adenocarcinoma (LUAD) tumors with their respective matched normal samples to define co-regulated phosphorylation modules within and across data sets. We show that putative modules can be used to nominate biomarkers for disease-specific treatments, as well as targets for novel treatment strategies, and highlight the strength of pan-cancer phosphoproteomics for cancer discovery.

EXPERIMENTAL PROCEDURES

Proteomics and Phosphoproteomics Data

The dataset we chose to demonstrate the utility of PhosphoDisco was collected and processed as part of the CPTAC consortium. Comprehensive characterization of the BRCA (17), LSCC (15), and LUAD (16) tumor and matched normal cohorts (Table 1) have been completed, and all samples were collected and processed according to the CPTAC standard protocols (20). Detailed experimental procedures including cohort statistics, clinical data, sample collection and processing, and data acquisition for these cohorts are described in detail elsewhere (15–17).

Briefly, tumor samples were snap-frozen less than 30 min after collection, after which genomic and transcriptomic sequencing was completed. Samples also underwent higher-energy C-trap dissociation (HCD) liquid chromatography (LC)-MS/MS analysis of tandem mass tag (TMT)-labeled samples for proteomic and phosphoproteomic characterization as previously described (20).

Data Processing and Quality Control

The Spectrum Mill software package v7.0 pre-release (Agilent Technologies, Santa Clara, CA) co-developed by Karl Clauser of the Carr laboratory (https://www.broadinstitute.org/proteomics) was used for MS data analysis. Protein identification was performed by searching the MS/MS spectra against the protein sequence database obtained using the UCSC Table Browser (https://genome.ucsc.edu/cgi-bin/hgTables) on September 14, 2016, which contains 37,579 proteins mapped to the human reference genome (hg19), adding common contaminants, mitochondrial proteins, and non-canonical small open reading frames. The searches were performed allowing ±20 ppm mass tolerance for precursor and product ions, allowing for common modification. Peptide spectrum matches (PSMs) were filtered for 30% minimum matched peak intensity and target-decoy-based false discovery rate (FDR) estimates at the PSM level, and for proteins protein level for each TMT-plex for all TMT-plexes for a tumor type, and for phosphorylation at the site levels. Normalization of each peptide was performed using the common reference, and a two-component Gaussian mixture model-based normalization was used to nullify the effect of differential protein loading and/or systematic MS variation.

PhosphoDisco Workflow

The PhosphoDisco workflow (supplemental Fig. S1 and supplemental Table S1) starts with normalizing, filtering, and pairwise correlation of peptide-level phosphorylation data, described in more detail below. The next steps are to find co-regulated modules relevant to clinical annotations, followed by nominating putative protein regulators for the modules. Further, Gene Ontology (GO) enrichment is then applied to find modules associated with annotated pathways, and motif analysis of phosphosite flanking sequences can identify common motifs across phosphosites within the module. Identification of kinase activation loop phosphosites, druggability analysis, and PTM set enrichment analysis is also applied to help rank nominated modules and their regulators (Fig. 1). Together, this approach enables the user to reduce complex phosphoproteomic data sets into potentially relevant signaling modules for further biological interrogation.

PhosphoDisco comes with a Snakemake (21) pipeline that can be run using phdc_run *via* the command line. Before running the pipeline, the user should generate a configuration file that includes the path to the phosphorylation and protein data files. Input and output structure, as well as example workflows can be found in the PhosphoDisco tutorial which comes with each PhosphoDisco installation. Although built to be run on a high-performance compute cluster, all the computation in this study can be performed on a laptop (20 GB RAM, Quad core Intel I7). All relevant code and documentation can be found here: https://github.com/ruggleslab/phosphodisco.

Input Data and Pre-processing

PhosphoDisco functions are built around the ProteomicsData class in Python, which requires phosphopeptide and protein input tables, assumed to be in the form of $\log_2$(relative abundance). Input data should be structured as samples as columns and proteins/phosphopeptides as rows (22). Detailed examples of both file structures can be found in supplemental Tables S2 and S3. We suggest that both tables be normalized for sample loading (*e.g.*, with median or upper quartile normalization). A function that can perform different normalization

Table 1
Dataset summary stats

Tumor type	Number of tumor samples	Number of normal samples	Number of phosphosites assigned to modules	Number of modules
LUAD	98	98	1684	14
LSCC	99	99	1684	14
BRCA	122	0	1017	69

Lists basic summary stats about the main data sets used in this paper. The two lung datasets (LSCC, LUAD) were used in a combined analysis, and included normal samples, while the breast dataset (BRCA) only included tumor samples.

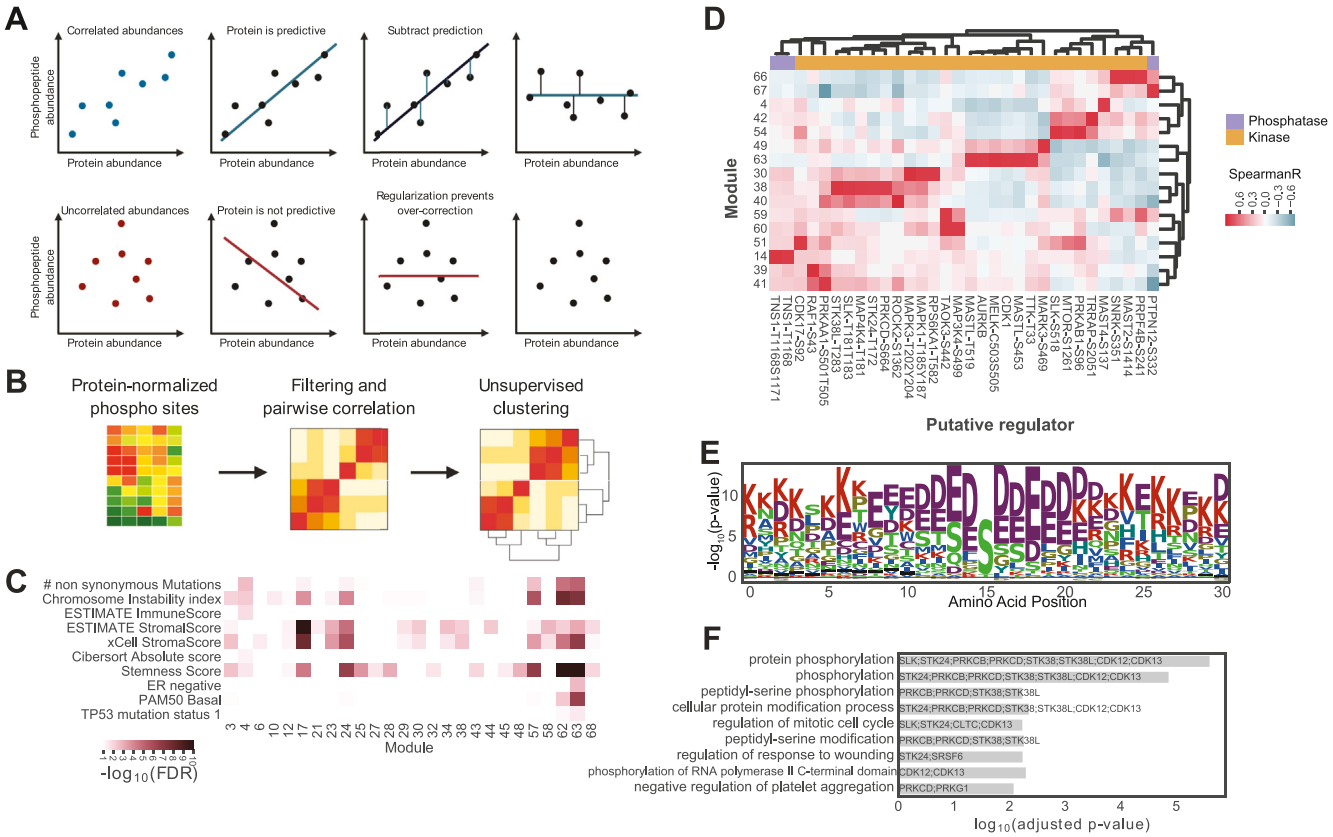

FIG. 1. **PhosphoDisco core functionalities.** *A*, regularized linear models are used to normalize phosphorylation data by protein abundance. *B*, normalized phosphorylation data are correlated with itself and modules are found by clustering the correlation matrix using hypercluster. *C*, clinical metadata are correlated with module scores to find relevant modules. *D*, kinase and phosphatase abundances are correlated with module scores and assigned potential regulators to modules. *E*, motifs can be calculated from peptides for a module. *F*, enrichments of phosphosites in a module can be calculated against a phosphosite annotations database like PhosphositePlus.

procedures is included (*column_normalize*) (supplemental Fig. S1). We do not recommend any row filtering and discourage row-normalizations like z-scoring, as maintaining different standard deviations between phosphosites is important for filtering before defining modules.

Protein Normalization of Phosphopeptides

Phosphopeptide relative abundance data can be difficult to interpret because differences in abundance can be due to changes either in parent protein abundance or differences in phosphorylation state. To account for these confounding factors, PhosphoDisco includes a *normalize_phospho_by_protein* method which accounts for variation in abundance of the parent protein, and extracts variation due to changes in phosphorylation. The main caveat of this approach is that it will over-correct for peptides that are auto-phosphorylated or auto-dephosphorylated on kinases and phosphatases, respectively. In these cases, protein abundance and phosphorylation are interconnected, and therefore normalizing by the former will cancel out the latter. For this reason, PhosphoDisco also identifies putative regulator sites on kinases and phosphatases which can identify these special cases (see section Association With Possible Regulators below).

To complete protein normalization for each phosphopeptide, we train a model using regularized linear regression, using cross-validation (CV) to choose the regularization parameter (*linear_regression.RidgeCV* from scikit-learn (23)). We train the model with phosphopeptide abundance as the target and its parent protein abundance as the feature. We then use this model to predict phosphopeptide abundance values based on the parent protein abundance and subtract that value from the phosphopeptide abundance (convert to residuals) (Fig. 1*A*), resulting in normalized phosphopeptide abundance (*ProteomicsData.normed_phospho*). During normalization, only peptides that share at least as many non-missing values as the CV fold are retained; this step acts as a missing values filter. Regularization is important in the case of low correlation between a phosphopeptide and its parent protein (*e.g.*, if a lack of variation in protein abundance across the cohort exists) (Fig. 1*A*). Regularization values and CV fold are changeable parameters, and by default, the regularization values 0.00032, 0.0016, 0.008, 0.04, 0.2, 1, 5, 25, 125, 625, and 3 CV folds are used.

When combining different datasets (*e.g.*, different cancer types or tumor and normal samples), we recommend first performing this normalization for each subgroup, so in the case of two cancer types A and B, and their matched normals, we would normalize A-normals, B-normals, A-tumors, and B-tumors independently. This helps ameliorate non-linear scale differences between datasets.

Filtering and Module Discovery

Prior to module discovery, we suggest that users retain only rows with high variance and low fraction of missing values from the protein normalized phosphopeptide table. This helps to reduce the required

memory needed to run the pipeline and our default filters out the lower 50% of variance rows, after filtering out rows with more than 25% of missing values. Both of these steps are automatically performed by the PhosphoDisco pipeline. If using more than one cohort (*e.g.*, multiple cancer types or tumor and normal samples) we recommend performing this filtering step within each subgroup separately. For example, in the case of two cancer types A and B, and their matched normals, we would apply these filters to the groups of A-normals, B-normals, A-tumors, B-tumors, A, B, as well as A and B combined (supplemental Fig. S3). We then keep the union of all phosphosites that pass our filters in any of the groups. In our analysis of a combined LSCC and LUAD phosphoproteomic dataset, we have observed that upwards of 90% of retained phosphosites in the A/B example would come from filtering A and B combined.

To assign modules, PhosphoDisco completes a pairwise correlation between each phosphosite using the *assign_modules* function. Given that there is no clear precedent for a clustering algorithm most suited for module discovery in proteomics and phosphoproteomics, one method is to run a multitude of clustering algorithms (23). Additionally, hyperparameters are adjustable options that a statistical algorithm exposes to the user, allowing them to modify how the algorithm operates. For clustering algorithms, it is often not obvious which configuration is best for addressing a given question for a given data set. The built-in hypercluster package (24) enables the execution of a combination of multiple clustering algorithms with a range of hyperparameter sets. Each of these hyperparameter sets then clusters the data into its own set of modules. Hypercluster offers a variety of different metrics for evaluating which module set to choose, including the adjusted Rand index (25), which can tell us which module set is the most inclusive, *i.e.*, the most similar to all other module sets.

For small datasets, the *assign_modules* function runs the default parameters of hypercluster.MultiAutoClusterer.fit to find modules. For large datasets we recommend finding optimized clusters using the hypercluster Snakemake (21) pipeline in a distributed manner, then using the final labels and evaluations to pick the best clustering method for module detection (Fig. 1*B*). In the examples below, we chose the best modules by finding the top parameter sets based on the highest adjusted Rand index as suggested to identify the most robust modules. An example table with modules can be found in supplemental Table S4. Once modules are defined, a score for each module is calculated with the *calculate_module_scores* method by taking the mean of all members of the module per sample. Next users can use the *impute_missing_values* function to impute values missing in the *normed_phospho* attribute of the ProteomicsData object. This can be useful for plotting purposes for data sets with few missing values but can lead to artifacts with larger missingness percentages. For this sklearn.impute is used with a KNNImputer by default.

PhosphoDisco includes a *visualize_modules* function that generates a clustered heatmap of protein-normalized phosphopeptides per sample for each co-regulated module (Fig. 2*B*). User-supplied sample annotations can be visualized alongside the heatmap to show relationships between module phosphosite levels and clinical features. This visualization is a key step for users to filter and interpret results. It is also important to visualize modules because clustering algorithms will often generate spurious modules, which can be manually removed by users from downstream analyses.

Association With Clinical Variables

Users can also assess whether module scores are associated with user-supplied sample annotations such as tumor subtype or survival using the *calculate_annotation_association* method. These associations are useful for finding modules that may be relevant to diagnostic subgroups of tumors. An example of such an annotation table can be found in supplemental Table S5. An annotations DataFrame can be added to a ProteomicsData object using the *add_annotations* method. When *add_annotations* is applied, users must supply a list, defining which columns of the DataFrame are continuous or categorical variables, as categorical columns are split into binarized groups. Next, using the *calculate_annotation_association* function, users can calculate *p*-values for module enrichment per group per categorical variables, or correlation for continuous variables. The *p*-values are then corrected for multiple hypothesis testing (default is Benjamini Hochberg procedure (26)) from the statsmodels package (27) (Fig. 1*C*).

Association With Possible Regulators

Proteins regulating the phosphorylation status of multiple phosphopeptides within a module are of particular interest as potential upstream targets for module inhibition and treatment. We therefore include methods to assist with nominating regulators for modules. Users can provide a list of gene names of putative regulators, matching identifiers in the top level of the index for the protein and phosphopeptide tables. The *collect_possible_regulators* function (supplemental Fig. S1 and supplemental Table S1) consolidates protein and phosphopeptide abundance data for all genes in that list. To prevent problems stemming from collinearity, this function also collapses features with higher than a user-set Pearson *R* value between them (default 0.95), by taking the mean values per sample. To prevent collapsing features, users can set the corr_threshold variable to a value above 1. After correlated features are collapsed, the remaining missing values are imputed with a sklearn.impute object (default is KNNImputer). The collected data is stored in the ProteomicsData.possible_regulators_data attribute. For this analysis, raw phosphopeptide data for each possible regulator is used, rather than protein-normalized data, to allow for the identification of auto-phosphorylation or auto-dephosphorylation, as discussed previously.

Once regulator data is collected, the *calculate_regulator_association* method finds the association between the vector of module scores and features in the possible_regulators_data attribute (Fig. 1*D*) for each pair of a regulator and a module. The method can be run in two modes: (a) it simply returns the correlation (default is Spearman), or (b) it builds a regularized linear model using possible_regulator_data as features and the modules scores at targets. There is also an option to transform the module scores so that the relationship being quantified between the regulators and modules scores is a sigmoid curve, as that is often the relationship between log(kinase abundance) and substrate concentration (28, 29) (supplemental Fig. S2*A*).

To aid in module prioritization, PhosphoDisco also provides the option to test the association of module scores with the abundances of a known set of kinase activation loop phosphosites (30) through the *correlate_kinase_activation_loop_phosphosites_with_module_scores* function. Kinase activation loop phosphorylation is strongly connected with kinase activity (30). Thus, this function can be used to further evidence for kinase activity beyond the more general *calculate_regulator_association* function. Users can also filter regulators by their known or predicted druggability using the druggable_regulator_heatmap function (31).

Motif Analysis

Phosphatases and kinases often show preference for specific amino acid motifs, therefore analyzing the peptide sequences within a module can help further refine our list of potential regulators. Flanking amino acid sequences of each peptide can be defined if users provide an additional DataFrame with one column containing the corresponding protein identifier appropriate for the given fasta file (*e.g.*, the RefSeq isoform ID for that genome version) and one column with a comma separated ordinal value of the modified amino acid(s) along with a corresponding protein fasta file. Users can employ the *collect_aa_sequences* method (supplemental Fig. S1 and supplemental

ASBMB